

Chapter 5

Video Coder Syntax

5.1. INTRODUCTION

The previous two chapters have dealt with the signal processing tools that are used to compress the amount of information needed to represent video material. The result of the signal processing is a series of primarily Huffman code words that represent the information that needs to be transmitted to the decoder in order to allow the reconstruction of the video service. Examples of the information that needs to be transmitted include quantized DCT coefficients represented as (run,level) pairs, motion vectors and quantizer step sizes. In addition, there is other information that needs to be transmitted to the decoder to allow successful decoding. This includes picture resolution (i.e., pixels per line and lines per picture), interlace structure, picture rate, and data rate.

This information needs to be delivered to the decoder in a form that the decoder can understand. This implies careful formatting and ordering to ensure that the information can be correctly interpreted when received by the decoder. In this chapter, we will look at the structure of the information that needs to be transmitted. This is followed by a consideration of the aspects that go to make a suitable syntax. Finally, we design a simple video encoder syntax that incorporates a number of the features found in the full syntax for MPEG-2 video. A knowledge of the fundamentals of video coder syntax greatly simplifies the process of understanding the complete MPEG-2 video standard, which is discussed in the following chapter.

5.2. REPRESENTATION OF CHROMINANCE INFORMATION

At its most general, each pixel in an image would be represented by a value for each of the primary colors that are red (R), green (G), and blue (B). However, as explained in Chapter 1, it is more usual to represent these three signals by a luminance (Y) signal that represents the gray scale level of the image and two chrominance signals (U and V). These are calculated according to

$$Y = 0.30R + 0.59G + 0.11B$$

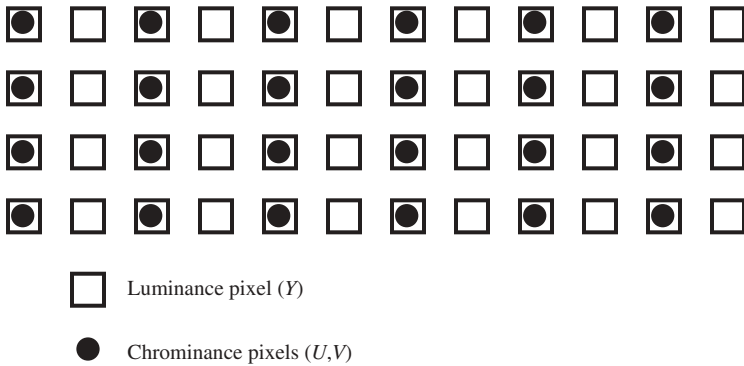


Figure 5.1 Luminance and chrominance positions in a 4:2:2 video picture.

$$U = \frac{B - Y}{2.03}$$

$$V = \frac{R - Y}{1.14}$$

If the R , G , and B components are sampled at CCIR Recommendation 601 resolution (i.e., 720×576 pixels at 25 Hz or 720×480 pixels at 30 Hz) then it is possible to have Y , U , and V components also represented at this resolution. However, the information content of the chrominance information is considerably less than that of the luminance information and thus some subsampling is possible. In CCIR Recommendation 601, the chrominance components are sampled horizontally at half the rate of the luminance information. The vertical chrominance sampling rate remains unchanged. This is known as the 4:2:2 video format and is shown in Figure 5.1.

The high chrominance accuracy of the 4:2:2 video format is important for specialized applications such as chromakeying.¹ However, for distribution television applications, further chrominance subsampling in the vertical direction is possible. The video format used in digital video broadcasting applications is usually the 4:2:0 format. The location of the luminance and chrominance pixels in the 4:2:0 video format are shown in Figure 5.2.

There is now exactly one pair of chrominance pixels for every four luminance pixels. Further, the chrominance pixels are located half way between each pair of horizontal lines. This can be achieved using a simple filtering procedure. This is explored further in a MATLAB exercise at the end of this chapter.

Figure 5.3 shows the position of the luminance and chrominance pixels in each field of an interlaced picture in 4:2:0 format.

¹An example of chromakeying is when a reporter stands in front of a blue screen that is replaced during postprocessing by some active video material. This is a common practice in the television industry.

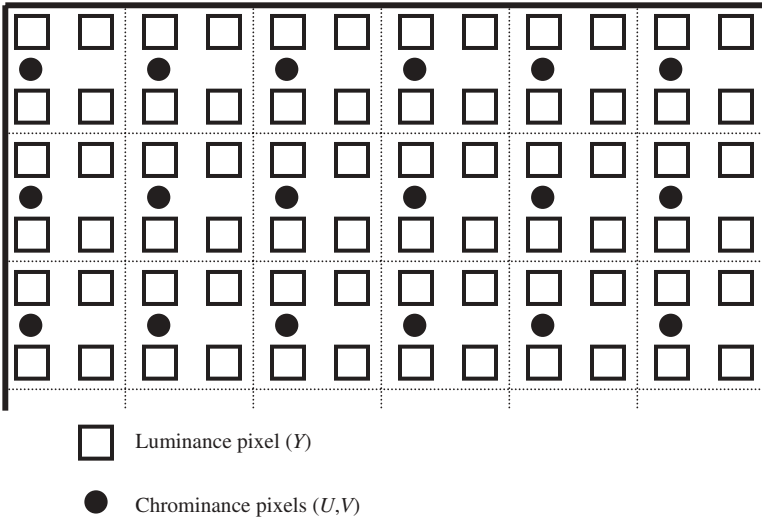


Figure 5.2 Luminance and chrominance positions in a 4:2:0 video picture.

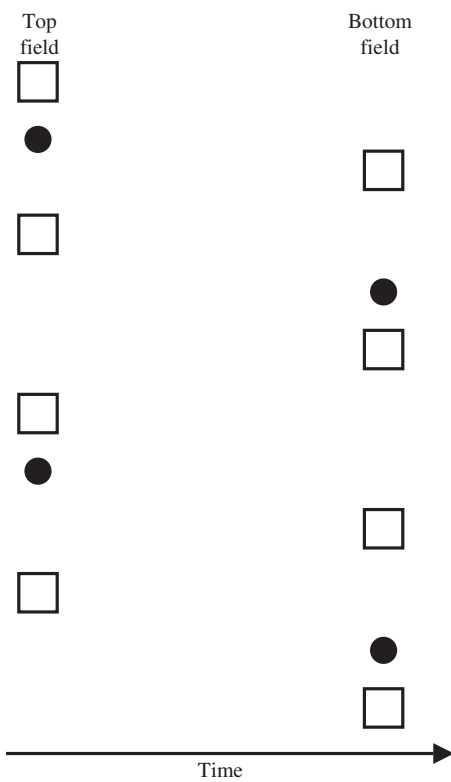


Figure 5.3 Vertical and temporal position of luminance and chrominance pixels in an interlaced 4:2:0 picture.

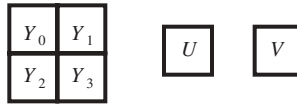


Figure 5.4 Blocks in a macroblock.

Note that in this case, the chrominance pixels do not lie vertically half way between the luminance pixels in a field. This approach means that the location of the chrominance pixels does not change depending upon whether the picture is coded as a whole (i.e., both fields at once) or one field at a time.

We saw in previous chapters that motion-compensated prediction is usually performed on blocks of size 16×16 pixels while the DCT is performed on blocks of 8×8 pixels. Using the 4:2:0 video format implies that for each 16×16 block of luminance pixels, there will be one 8×8 block of each of the two chrominance pixels. There will therefore be a total of six 8×8 pixel blocks, four luminance (Y_0 , Y_1 , Y_2 , and Y_3), and two chrominance (U and V), to be coded using the DCT as shown in Figure 5.4. Each 8×8 group of pixels (whether luminance or chrominance) is usually called a *block*.

5.3. STRUCTURE OF A VIDEO BIT STREAM

Video encoder syntax is made up of a hierarchical tree of layers starting with the sequence layer at the top and finishing at the block layer at the bottom. This is shown in Figure 5.5.

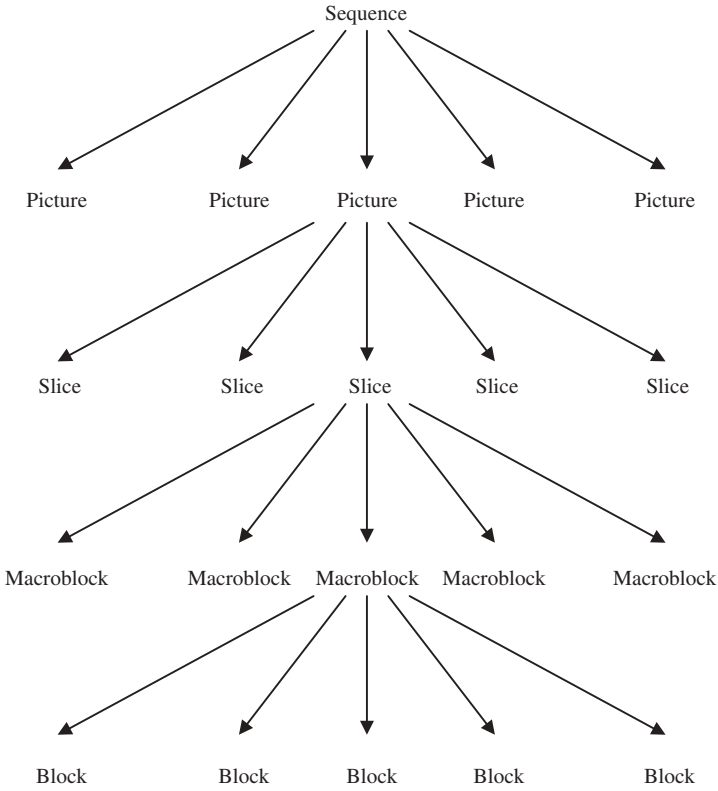
So far, we have considered how blocks of quantized DCT coefficients are zig-zag scanned and then transmitted in a bit stream. We now study how we move from the coding of blocks of picture data to the coding of an entire sequence. For simplicity, we start at the level at which we are familiar, namely the block level, and then move up to the high levels in the hierarchy.

5.3.1. The Block Layer

A video encoder output bit stream is generated in a bottom up manner starting from the simplest element namely a Huffman code word that represents a single (run,coefficient) pair or, as they are more usually referred to in the MPEG standards, a (run,level) pair corresponding to a single nonzero DCT coefficient. A number of these (run,level) pairs, together with an end of block (EOB) code word, combine to represent an 8×8 block of quantized DCT coefficients. This structure is usually referred to as a block.

5.3.1.1. Quantization of DCT Coefficients

With the exception of the DC coefficient in intracoded macroblocks (which is discussed a little later), we assume that quantization is carried out according to the

**Figure 5.5** Bit-stream structure.

formula given below.

$$\hat{C}(i, j) = \frac{C(i, j)}{8 \times 2 \times \text{quantizer_scale}}$$

where $C(i, j)$ is the original DCT coefficient value, $\hat{C}(i, j)$ is the quantized DCT coefficient value rounded to the nearest integer, and quantizer_scale is a value extracted from the bit stream that indicates the current quantizer step size.

5.3.1.2. Coding of (run,level) Pairs of DCT Coefficients

Each of the (run,level) pairs associated with each nonzero DCT coefficient is encoded using a variable length code word and this leads to a problem. Runs of zeros are in the range 0–63 (at total of 64 possibilities) while the level can be in the range ± 2047 (4094 possibilities²) for the video standard used for digital television

²A level of 0 cannot occur as all zero coefficients would be incorporated in to the run part of the (run,level) pair.

broadcasting. This leads to a total of 262,016 (64×4094) different variable length code words. Hardware capable of decoding such a huge number of variable length code words would be excessively complex. On the contrary, using fixed length code words would require at least 18 bits per code word (6 bits for run and 12 bits for level information in 2's complement form), and this is also highly inefficient. A compromise approach is therefore employed. Variable length code words are used for more common (run,level) pairs, with fixed length code words used for the remaining majority (in terms of number but not in terms of likelihood) of (run,level) pairs. Of course, the decoder needs to be able to determine whether a particular code word is from the variable or fixed length code word set. In our syntax, each fixed length code word is therefore preceded by a 6-bit ESCAPE code that is unique among the variable length code words. Upon receipt of an escape code, the decoder knows that the next 18 bits carry a fixed length code specifying the (run,level) pair.

A typical set of variable length codes for the coding of (run,level) pairs is given in Table 5.1, whereas the fixed length code words that apply only for those (run,level) pairs not included in Table 5.1 are shown in Figure 5.6. The code words in Table 5.1 deal with both positive and negative level values. In the case of a positive value of level, the last bit of the code word (indicated by an "x") is a 0 whereas for a negative value of level it is a 1. Thus a (0,+1) (run,level) pair is indicated by the code word 110 whereas a (0,-1) (run,level) pair is indicated by the code word 111.

For the variable length code words, as might be intuitively expected, the length of the code word increases as either the length of the run of zeros or the absolute level increases as each of these makes that particular (run,level) pair less likely. No valid variable length code word is ever a prefix for any other valid code word as expected with Huffman codes. The longest code word in the variable length code words is of length 17 bits compared to the 24 bits required when an ESCAPE code word is needed.

5.3.1.3. Block with All Zero DCT Coefficients

Successful motion-compensated prediction combined with quantization of the resulting DCT coefficients can often lead to a situation where all of the DCT coefficients in a block are quantized to zero. Using the variable length codes shown in Table 5.1, such a block would be coded with an end of block code word that requires two bits. A more efficient approach is to simply not transmit any information for this block—such a block is called a *skipped block*. However, this immediately leads to a problem, because the decoder needs to know which blocks have been coded and which blocks have been skipped. This is achieved by placing information in the header of each macroblock that indicates which blocks within the macroblock are actually coded (i.e., contain at least one nonzero quantized DCT coefficient).

5.3.2. The Macroblock Layer

A macroblock comprises a 16×16 pixel block of luminance pixels together with the associated chrominance information. For the 4:2:0 video format, there would

Table 5.1 Table of variable length codes for the coding of (run,level) pairs of DCT coefficients.

Run	Level	Code word	Run	Level	Code word
0	1	11x	0	4	0000 110x
1	1	011x	1	4	0000 0011 00x
2	1	0101 x	2	4	0000 0001 0100 x
3	1	0011 1x	3	4	0000 0000 1001 1x
4	1	0011 0x	0	5	0010 0110 x
5	1	0001 11x	1	5	0000 0001 1011 x
6	1	0001 01x	2	5	0000 0000 1010 0x
7	1	0001 00x	0	6	0010 0001 x
8	1	0000 111x	1	6	0000 0000 1011 0x
9	1	0000 101x	0	7	0000 0010 10x
10	1	0010 0111 x	1	7	0000 0000 1010 1x
11	1	0010 0011 x	0	8	0000 0001 1101 x
12	1	0010 0010 x	1	8	0000 0000 0011 111x
13	1	0010 0000 x	0	9	0000 0001 1000 x
14	1	0000 0011 10x	1	9	0000 0000 0011 110x
15	1	0000 0011 01x	0	10	0000 0001 0011 x
16	1	0000 0010 00x	1	10	0000 0000 0011 101x
17	1	0000 0001 1111 x	0	11	0000 0001 0000 x
18	1	0000 0001 1010 x	1	11	0000 0000 0011 100x
19	1	0000 0001 1001 x	0	12	0000 0000 1101 0x
20	1	0000 0001 0111 x	1	12	0000 0000 0011 011x
21	1	0000 0001 0110 x	0	13	0000 0000 1100 1x
22	1	0000 0000 1111 1x	1	13	0000 0000 0011 010x
23	1	0000 0000 1111 0x	0	14	0000 0000 1100 0x
24	1	0000 0000 1110 1x	1	14	0000 0000 0011 001x
25	1	0000 0000 1110 0x	0	15	0000 0000 1011 1x
26	1	0000 0000 1101 1x	1	15	0000 0000 0001 0011 x
27	1	0000 0000 0001 1111 x	0	16	0000 0000 0111 11x
28	1	0000 0000 0001 1110 x	1	16	0000 0000 0001 0010x
29	1	0000 0000 0001 1101 x	0	17	0000 0000 0111 10x
30	1	0000 0000 0001 1100 x	1	17	0000 0000 0001 0001 x
31	1	0000 0000 0001 1011 x	0	18	0000 0000 0111 01x
0	2	0100 x	1	18	0000 0000 0001 0000 x
1	2	0001 10x	0	19	0000 0000 0111 00x
2	2	0000 100x	0	20	0000 0000 0110 11x
3	2	0010 0100 x	0	21	0000 0000 0110 10x
4	2	0000 0011 11x	0	22	0000 0000 0110 01x
5	2	0000 0010 01x	0	23	0000 0000 0110 00x
6	2	0000 0001 1110 x	0	24	0000 0000 0101 11x
7	2	0000 0001 0101 x	0	25	0000 0000 0101 10x
8	2	0000 0001 0001 x	0	26	0000 0000 0101 01x

(continued)

Table 5.1 (Continued)

Run	Level	Code word	Run	Level	Code word
9	2	0000 0000 1000 1x	0	27	0000 0000 0101 00x
10	2	0000 0000 1000 0x	0	28	0000 0000 0100 11x
11	2	0000 0000 0001 1010 x	0	29	0000 0000 0100 10x
12	2	0000 0000 0001 1001 x	0	30	0000 0000 0100 01x
13	2	0000 0000 0001 1000 x	0	31	0000 0000 0100 00x
14	2	0000 0000 0001 0111 x	0	32	0000 0000 0011 000x
15	2	0000 0000 0001 0110 x	0	33	0000 0000 0010 111x
16	2	0000 0000 0001 0101 x	0	34	0000 0000 0010 110x
0	3	0010 1x	0	35	0000 0000 0010 101x
1	3	0010 0101 x	0	36	0000 0000 0010 100x
2	3	0000 0010 11x	0	37	0000 0000 0010 011x
3	3	0000 0001 1100 x	0	38	0000 0000 0010 010x
4	3	0000 0001 0010 x	0	39	0000 0000 0010 001x
5	3	0000 0000 1001 0x	0	40	0000 0000 0010 000x
6	3	0000 0000 0001 0100 x			
End of block	10		ESCAPE	0000 01	

© This Table is based on AS/NZS 13818.2:2002. Permission to reprint has been granted by SAI Global Ltd. The standard can be purchased online at <http://www.sai-global.com>.

be four luminance blocks and two chrominance blocks (one of U pixels and one of V pixels) in each macroblock. This was shown in Figure 5.4. Because motion-compensated prediction is performed on 16×16 pixel blocks, each macroblock also contains a motion vector if motion-compensated prediction is employed.

Escape code word				
0000 01	Run	Code word	Level	Code word
	0	0000 00	-2047	1000 0000 0001
	1	0000 01	-2046	1000 0000 0010
	2	0000 10	-2045	1000 0000 0011
	...	...	...	...
	...	...	...	...
	...	...	...	...
	...	...	-2	1111 1111 1110
	...	...	-1	1111 1111 1111
	...	...	0	Not allowed
	63	1111 11	+1	0000 0000 0001
			+2	0000 0000 0010
			...	...
			...	...
			...	...
			+2045	0111 1111 1101
			+2046	0111 1111 1110
			+2047	0111 1111 1111

Figure 5.6 Table of fixed length codes for the coding of (run,level) pairs of DCT coefficients.

© This Table is based on AS/NZS 13818.2:2002. Permission to reprint has been granted by SAI Global Ltd. The standard can be purchased online at <http://www.sai-global.com>.

5.3.2.1. Macroblock Type

Each macroblock can be coded in one of the following three modes: intramode, intermode, or motion-compensated prediction mode. The mode is used to inform the decoder which type of macroblock is being transmitted. In intramode, the macroblock is coded without temporal prediction from an earlier picture. For intermode, the macroblock is coded using temporal prediction by the macroblock in exactly the same position in an earlier (prediction) picture. In the motion-compensated prediction mode, the macroblock is coded using temporal prediction with a macroblock-sized block of data in an earlier (prediction) picture indicated by a motion vector.

Intermode is simply motion-compensated mode with a motion vector of (0,0). However, indicating intermode means that there is no requirement to transmit the motion vector. This results in an overall saving in the total number of bits required to represent the macroblock.

It is also possible to change the value of the quantizer step size at the macroblock level. Again, the decoder needs to know that a new quantizer is about to be specified. All of this information is contained in the macroblock header in a field called the *macroblock type*. An example of a typical macroblock type field is shown in Table 5.2.

The macroblock type information is variable length coded to minimize the overhead associated with its transmission. Because every block in an intracoded block requires at least a DC DCT coefficient, every block in an intra macroblock requires coding. In an intercoded macroblock, if all of the quantized DCT coefficients in each block are zero then the macroblock does not need to be transmitted—such a macroblock is called a *skipped macroblock*. Any skipped macroblocks are decoded as the reconstructed macroblock in the same spatial location in the reference picture. As with skipped blocks, there is a need to signal to the decoder the location of the current macroblock in case one or more of the macroblocks that immediately precedes it have been skipped. How this is achieved is described in detail a little later.

5.3.2.2. Macroblock Address

As for skipped blocks, it is also possible for a video encoder to skip an entire macroblock. In this case, the macroblock is assumed to have been encoded using interpicture prediction (i.e., the macroblock is predicted by the macroblock of information in the

Table 5.2 Macroblock type field.

Macroblock mode	Same quantizer	New quantizer	Not coded
Intra	0001 1	0000 01	Not allowed
Inter	01	0000 1	Skipped
Motion compensated	1	001	0010 0

same position in the reference picture) and all quantized DCT coefficients from this prediction difference are zero. The result is that the reconstructed macroblock in the current picture is simply the reconstructed macroblock in the same position in the reference picture. As with skipped blocks, the decoder needs to be able to detect when a macroblock has been skipped. One way that this could be achieved would be to number macroblocks from left to right across a picture. For a CCIR Recommendation 601 picture with 704 pixels/lines, there would be 44 macroblocks in a row of macroblocks thus requiring a 6-bit address field in the macroblock header.

In fact rather than transmitting the macroblock address, it is more usual to transmit the difference between the current macroblock and the last macroblock that was transmitted. This is called the *macroblock address increment*. Thus if the current macroblock is macroblock 3 and the last transmitted macroblock was macroblock 2 then the macroblock address increment would have value 1. Not all macroblock address increments are equally likely, and so variable length coding is once again possible. Table 5.3 shows typical variable length code words for the macroblock address increment. It is clear that small macroblock address increments are the most probable.

The **macroblock_escape** code word indicates that the macroblock address increment is greater than 33. It causes the value 33 to be added each time it occurs. Thus a macroblock address increment of 67 would be represented by the code word shown in Figure 5.7.

Table 5.3 Typical Huffman codes for macroblock address increment.

Macroblock address increment	Code word	Macroblock address increment	Code word
1	1	18	0000 0101 01
2	011	19	0000 0101 00
3	010	20	0000 0100 11
4	0011	21	0000 0100 10
5	0010	22	0000 0100 011
6	0001 1	23	0000 0100 010
7	0001 0	24	0000 0100 001
8	0000 111	25	0000 0100 000
9	0000 110	26	0000 0011 111
10	0000 1011	27	0000 0011 110
11	0000 1010	28	0000 0011 101
12	0000 1001	29	0000 0011 100
13	0000 1000	30	0000 0011 011
14	0000 0111	31	0000 0011 010
15	0000 0110	32	0000 0011 001
16	0000 0101 11	33	0000 0011 000
17	0000 0101 10	macroblock_escape	0000 0001 000

© This Table is based on AS/NZS 13818.2:2002. Permission to reprint has been granted by SAI Global Ltd. The standard can be purchased online at <http://www.sai-global.com>.

```

0000 0001 000      0000 0001 000      1
macroblock_escape macroblock_escape      macroblock address increment = 1

```

Figure 5.7 Code word for a macroblock address increment of 67.

This is another example of a modified Huffman code.

5.3.2.3. Coding of Motion Vectors

Each macroblock that is coded in motion-compensated mode requires a motion vector to be included in its header information. This motion vector indicates the position in the reference picture of the macroblock that is to be used to predict the current macroblock. A positive value of the horizontal component of the motion vector means that the prediction is made from pixels in the reference picture that lie to the right of the pixels that are being predicted. Similarly, a positive value of the vertical component of the motion vector means that the prediction is made from pixels in the reference picture that lie below the pixels that are being predicted.

For a motion vector range of -16 to $+15$ both horizontally and vertically, five bits would be needed to represent each motion vector—a total of 10 bits per macroblock. Fortunately, motion vector fields are often highly correlated. Figure 5.8

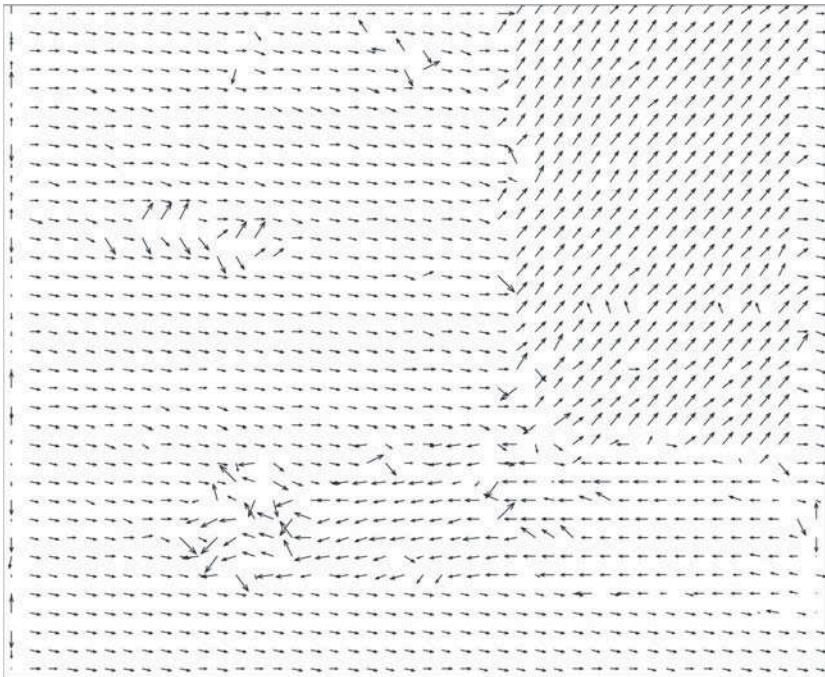


Figure 5.8 Motion vector field for first picture of “Mobile and Calendar” sequence.

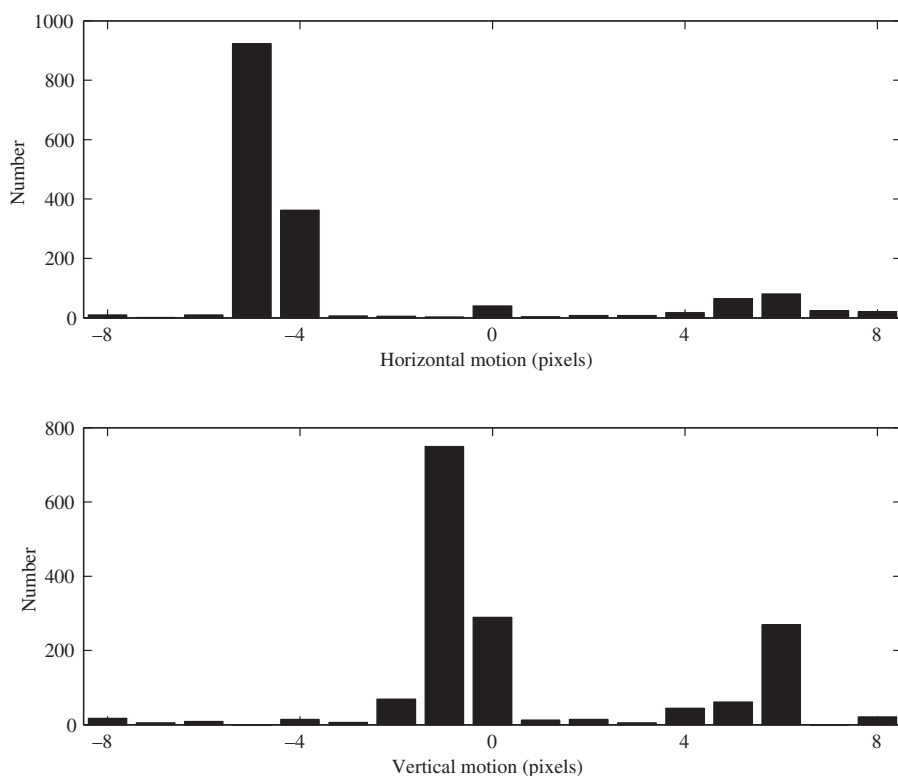


Figure 5.9 Histogram of horizontal and vertical motion components for Figure 5.8.

shows the motion vectors for a picture of the “Mobile and Calendar” sequence. The similarity in nearby motion vectors is clearly apparent. This is especially true in the calendar and also in the wallpaper background.

Figure 5.9 shows the histogram for the horizontal and vertical component of each motion vector. There are two dominant motions in the horizontal direction (-4 pixels and -5 pixels) corresponding to the motion in the background wallpaper. In the vertical direction, there is a pair of dominant motions close to zero (-1 pixels and 0 pixels) again corresponding to motion in the background and another dominant motion (at $+6$ pixels) corresponding to the motion of the calendar.

It is clear that there is strong correlation between nearby motion vectors. This correlation is exploited using lossless one-dimensional horizontal prediction of each component of the motion vector. Thus each component of the motion vector is predicted by the identical component of the motion vector immediately to its left and only the prediction difference is sent to the decoder. The first pixel is transmitted without prediction. If the first motion vector in a row was $(+3, -2)$ and the second motion vector was $(+2, -3)$, then the prediction difference that would be sent is $(+2 - +3, -3 - -2) = (-1, -1)$. Histograms of the prediction difference for the

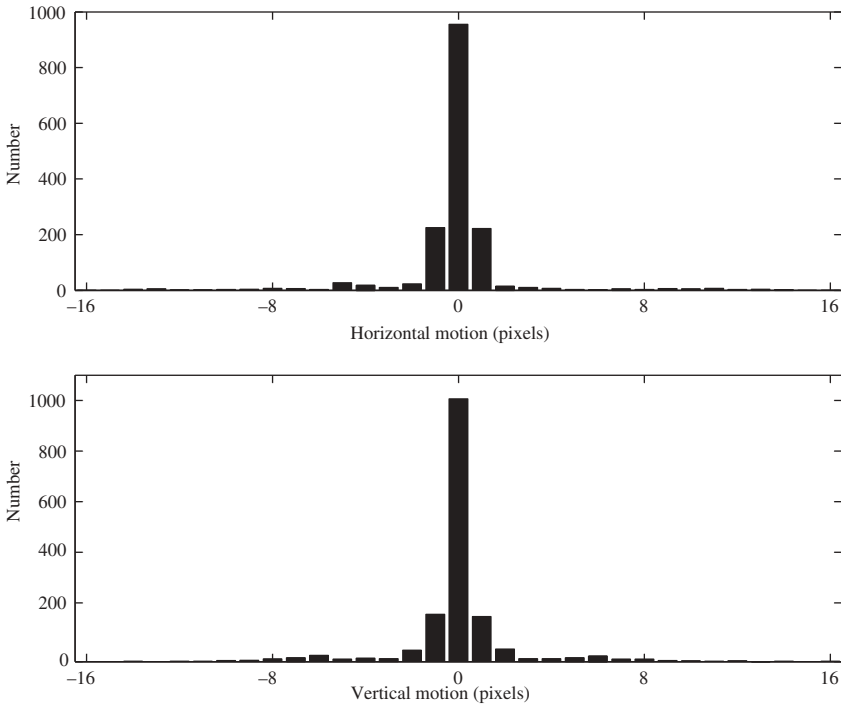


Figure 5.10 Histogram motion vector prediction differences for Figure 5.8.

motion vector field shown in Figure 5.8 are shown in Figure 5.10. Note that for motion vector components in the range of ± 8 , prediction differences will be in the range of ± 16 .

It is clear from Figure 5.10 that the prediction differences are closely packed around zero. A worthwhile saving therefore results if the prediction differences are coded using a variable length Huffman code. For the example just completed, the number of bits per motion vector component is reduced from 4 bits to approximately 2 bits—a saving of 50%.

A typical Huffman code for the motion vector prediction differences in the range of ± 16 is shown in Table 5.4. Again, the last bit of the variable length code word indicates the sign of the motion vector prediction difference. In the case of a positive difference, the last bit of the code word (indicated by an “x”) is 0 whereas for a negative value of the difference is 1. As expected, the code word becomes longer as the absolute size of the prediction difference increases.

5.3.2.4. Coding of DC Coefficients in Intramacroblocks

In intracoded macroblocks, the DC DCT coefficient always needs to be coded. In addition, the DC value of an intracoded block is particularly important as it represents the average luminance or chrominance value of the block. Significant quantization

Table 5.4 Typical motion vector prediction difference Huffman codes.

Motion vector prediction difference	Code word
0	1
1	01x
2	001x
3	0001 x
4	0000 11x
5	0000101x
6	0000 100x
7	0000 011x
8	0000 0101 1x
9	0000 0101 0x
10	0000 0100 1x
11	0000 0100 01x
12	0000 0100 00x
13	0000 0011 11x
14	0000 0011 10x
15	0000 0011 01x
16	0000 0011 00x

© This Table is based on AS/NZS 13818.2:2002. Permission to reprint has been granted by SAI Global Ltd. The standard can be purchased online at <http://www.sai-global.com>.

of this coefficient would be readily apparent in the decoded picture because it would lead to sharp intensity changes between adjacent blocks. These would show up as blocking artifacts (i.e., block boundaries would be clearly visible in the decoded picture). For this reason, the DC coefficient in intramacroblocks is not coded using the quantizer that is applied to the other coefficients in the block. Instead, a quantizer step size of 8 is always applied. Thus a DC coefficient of 601 would be quantized to a value of 75 irrespective of the quantizer used for the remainder of the macroblock.

In fact, the intra DC coefficients are coded separately from the other DCT coefficients that are coded as (run,level) pairs as shown in Table 5.1. We have seen already that there is strong correlation between nearby pixels in images. For this reason, there is also strong correlation between the DC DCT coefficients of nearby blocks within a picture because the DC value represents the average luminance or chrominance value of that block in the picture. Figure 5.11 shows the average luminance value of each 8×8 pixel block within the first picture of the “Mobile and Calendar” sequence.

An approach similar to that employed to code motion vectors can be used to code the DC DCT coefficients in intramacroblocks. For simplicity, prediction is made based on blocks in the current macroblock or from the previous macroblock as shown in Table 5.5. The block nomenclature shown in Figure 5.4 is used in Table 5.5.

When this lossless prediction is applied to the luminance data of the first picture of the “Mobile and Calendar” sequence, the prediction differences produce the

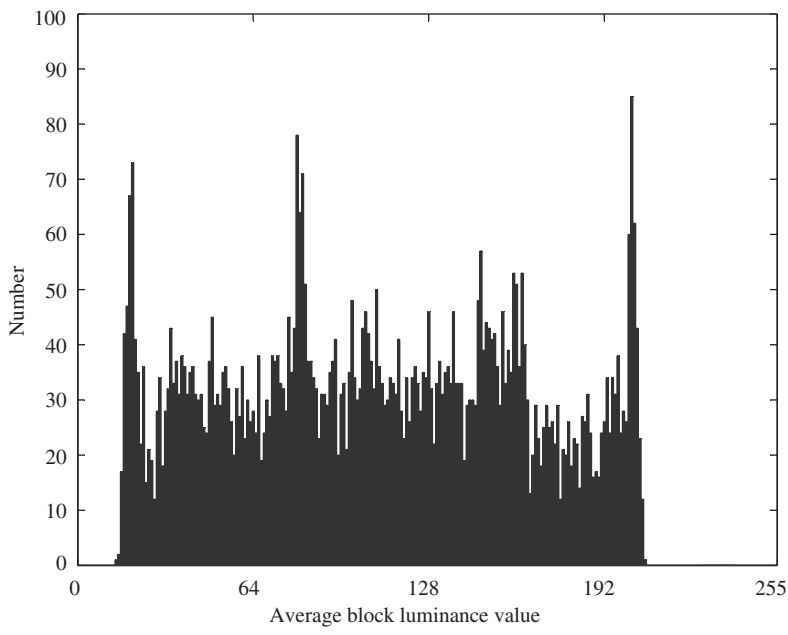


Figure 5.11 Histogram of average luminance values.

histogram shown in Figure 5.12. It is clear that some saving is possible using an appropriate variable length Huffman code. Note also that blocks used for prediction are all within the current row of macroblocks. This means that a row of macroblocks can be correctly decoded even if the previous row of macroblocks could not be decoded due to transmission errors, for example. The predictor is reset to 1024 (128×8) at the beginning of each row of macroblocks.

In fact the difference information is coded using two code words. The first code word specifies the length in bits of the difference (as shown in Table 5.6) whereas the second code word specifies the actual value (as shown in Table 5.7). Note that the code words used for length in bits are different for luminance and chrominance.

Table 5.5 Prediction of DC DCT coefficient in intramacroblocks.

Block	Prediction block
Y_0	Y_3 in previous macroblock
Y_1	Y_0 in current macroblock
Y_2	Y_1 in current macroblock
Y_3	Y_2 in current macroblock
U	U in previous macroblock
V	V in previous macroblock

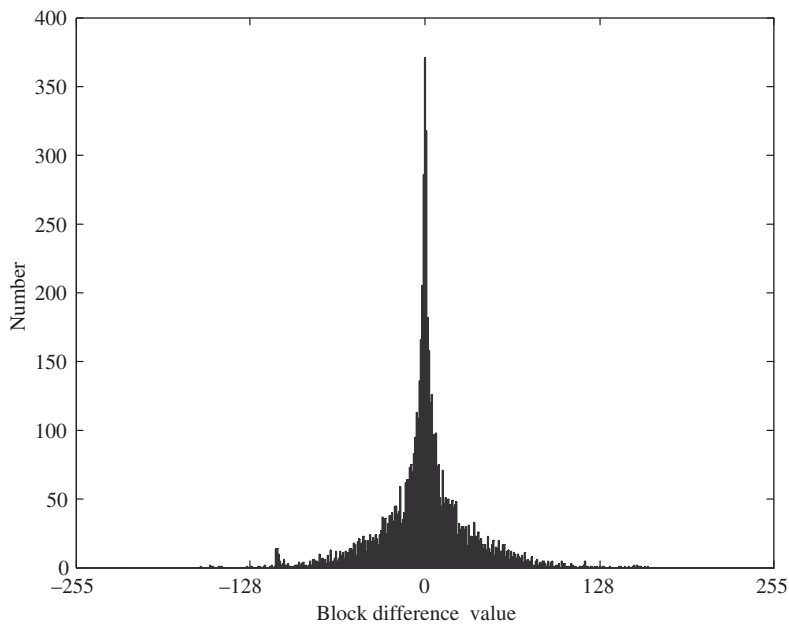


Figure 5.12 Histogram of DC block differences.

Thus a prediction difference of 29 in the luminance DC DCT value would be represented by the code words given in Figure 5.13.



Figure 5.13 Calculation of luminance DC DCT code-word.

Table 5.6 Variable length codes for the differential DC length.

Absolute value of differential DC	Length (bits)	VLC code (luminance)	VLC code (chrominance)
0	0	100	00
1	1	00	01
2–3	2	01	10
4–7	3	101	110
8–15	4	110	1110
16–31	5	1110	1111 0
32–63	6	1111 0	1111 10
64–127	7	1111 10	1111 110
128–255	8	1111 110	1111 1110

© This Table is based on AS/NZS 13818.2:2002. Permission to reprint has been granted by SAI Global Ltd. The standard can be purchased online at <http://www.sai-global.com>.

Table 5.7 Value of differential DC.

Differential DC	Length (bits)	Code word
−255 to −128	8	00000000 to 01111111
−127 to −64	7	0000000 to 0111111
−63 to −32	6	000000 to 011111
−31 to −16	5	00000 to 01111
−15 to −8	4	0000 to 0111
−7 to −4	3	000 to 011
−3 to −2	2	00 to 01
−1	1	0
0	0	
1	1	1
2–3	2	10 to 11
4–7	3	100 to 111
8–15	4	1000 to 1111
16–31	5	10000 to 11111
32–63	6	100000 to 111111
64–127	7	1000000 to 1111111
128–255	8	10000000 to 11111111

© This Table is based on AS/NZS 13818.2:2002. Permission to reprint has been granted by SAI Global Ltd. The standard can be purchased online at <http://www.sai-global.com>.

5.3.2.5. Coded Block Pattern

As indicated when we were considering the block level, it is not necessary to code every block of DCT coefficients within a macroblock because some blocks, after quantization, will have all zero coefficients. In this case, the block is skipped. The decoder needs to know which blocks within a macroblock have been skipped. This is signaled in the macroblock header by a field called the coded block pattern (CBP). The value of the CBP is determined as follows with reference to Figure 5.4: If a block requires coding, that block is given the value 1 in the CBP. If the block is skipped, that block is given the value 0. Then

$$\text{CBP} = 32 \times Y_1 + 16 \times Y_2 + 8 \times Y_3 + 4 \times Y_4 + 2 \times U + 1 \times V$$

Thus, if all blocks needed to be coded then CBP would take the value 63 whereas CBP would take the value 60 if only the luminance blocks are needed to be coded. Some values of CBP are more likely than others, and so a saving can be achieved using Huffman coding. A typical Huffman code table for the CBP is shown in Table 5.8. Note that the CBP takes values in the range 1–63. A CBP of zero would imply either a skipped macroblock (i.e., no data is transmitted about the macroblock) or a motion-compensated not coded macroblock type (see Table 5.2) in which case a CBP is not transmitted.

These Huffman code words have been developed based on the statistics of real coded video sequences. The shortest code word and thus the highest probability symbol (CBP = 60) corresponds to the case of all four luminance blocks coded but

Table 5.8 Huffman code words for coded block pattern.

Coded block pattern	Pattern[i]						Code word
	$Y_0 (i=0)$	$Y_1 (i=1)$	$Y_2 (i=2)$	$Y_3 (i=3)$	$U (i=4)$	$V (i=5)$	
1						1	0101 1
2					1		0100 1
3					1	1	0011 01
4				1			1101
5				1		1	0010 111
6				1	1		0010 011
7				1	1	1	0001 1111
8			1				1100
9			1			1	0010 110
10			1		1		0010 010
11			1		1	1	0001 1110
12			1	1			1001 1
13			1	1		1	0001 1011
14			1	1	1		0001 0111
15			1	1	1	1	0001 0011
16		1					1011
17		1				1	0010 101
18		1			1		0010 001
19		1			1	1	0001 1101
20		1		1			1000 1
21		1		1		1	0001 1001
22		1		1	1		0001 0101
23		1		1	1	1	0001 0001
24		1	1				0011 11
25		1	1			1	0000 1111
26		1	1		1		0000 1101
27		1	1		1	1	0000 0001 1
28		1	1	1			0111 1
29		1	1	1		1	0000 1011
30		1	1	1	1		0000 0111
31		1	1	1	1	1	0000 0011 1
32	1						1010
33	1					1	0010 100
34	1				1		0010 000
35	1				1	1	0001 1100
36	1			1			0011 10
37	1			1		1	0000 1110
38	1			1	1		0000 1100
39	1			1	1	1	0000 0001 0
40	1		1				1000 0
41	1		1			1	0001 1000

(continued)

Table 5.8 (Continued)

Coded block pattern	Pattern[i]						Code word
	$Y_0 (i=0)$	$Y_1 (i=1)$	$Y_2 (i=2)$	$Y_3 (i=3)$	$U (i=4)$	$V (i=5)$	
42	1		1		1		0001 0100
43	1		1		1	1	0001 0000
44	1		1	1			0111 0
45	1		1	1		1	0000 1010
46	1		1	1	1		0000 0110
47	1		1	1	1	1	0000 0011 0
48	1	1					1001 0
49	1	1				1	0001 1010
50	1	1			1		0001 0110
51	1	1			1	1	0001 0010
52	1	1		1			0110 1
53	1	1		1		1	0000 1001
54	1	1		1	1		0000 0101
55	1	1		1	1	1	0000 0010 1
56	1	1	1				0110 0
57	1	1	1			1	0000 1000
58	1	1	1		1		0000 0100
59	1	1	1		1	1	0000 0010 0
60	1	1	1	1			111
61	1	1	1	1		1	0101 0
62	1	1	1	1	1		0100 0
63	1	1	1	1	1	1	0011 00

© This Table is based on AS/NZS 13818.2:2002. Permission to reprint has been granted by SAI Global Ltd. The standard can be purchased online at <http://www.sai-global.com>.

with both chrominance blocks skipped. Next most likely are macroblocks containing just a single coded luminance block ($CBP = 32, 16, 8,$ or 4) followed by macroblocks containing just two coded luminance blocks. It is clear that skipped blocks occur regularly in coded video sequences.

5.3.2.6. Summary

The macroblock header contains a large amount of information required to allow the decoder to correctly interpret the information contained in the macroblock. The information contained in the macroblock header is a function of the type of macroblock that is being coded. As shown in Table 5.2, there are seven valid macroblock types.³ The format of each of these macroblock types is shown in Figure 5.14. For

³As we shall see in the next chapter, there are a considerably larger number of macroblock types available in the compression standard used for digital video broadcasting.

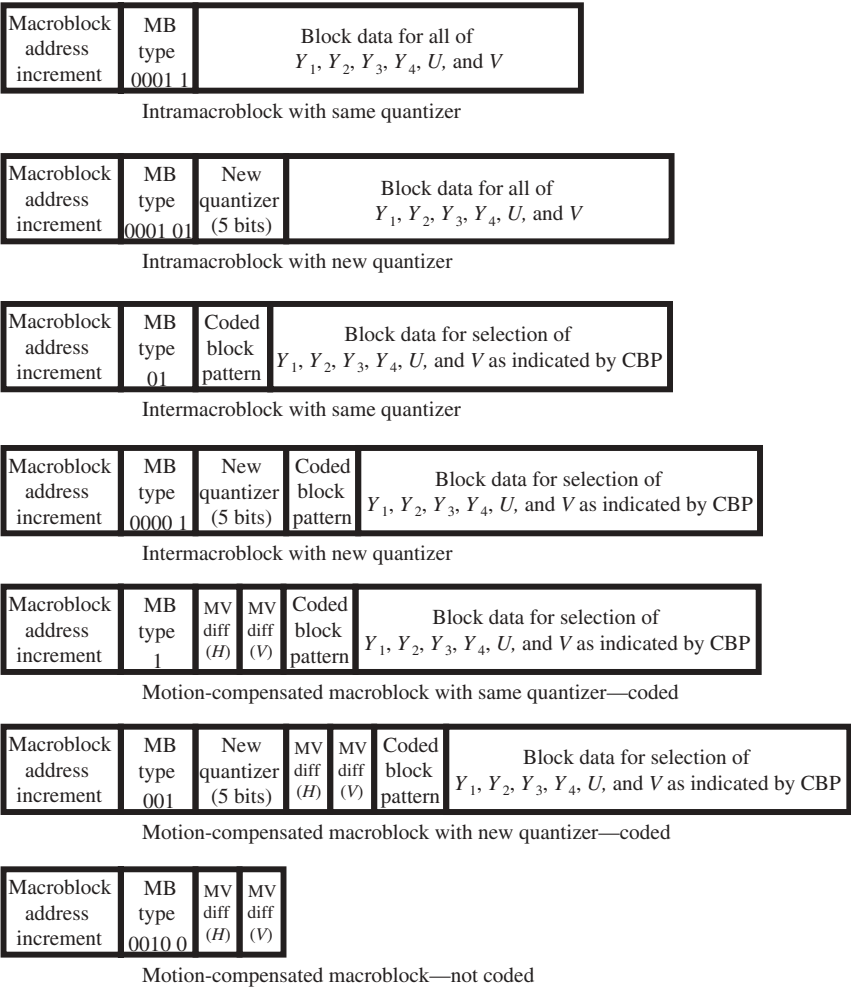


Figure 5.14 Format of various macroblock types.

intramacroblocks, each block is always coded as it will contain at least a DC DCT coefficient. In motion-compensated macroblocks, MV Diff (H) and MV Diff (V) are the horizontal and vertical difference motion vectors respectively.

5.3.3. The Slice Layer

Macroblocks merge, commonly, to form a slice with a slice containing a complete row of macroblocks as shown in Figure 5.15. In this case, n macroblocks in the same row merge to form a slice.

Each slice needs a header. One important task of the slice header is to allow for decoder resynchronization in case the decoder reaches a stage where it is unable to

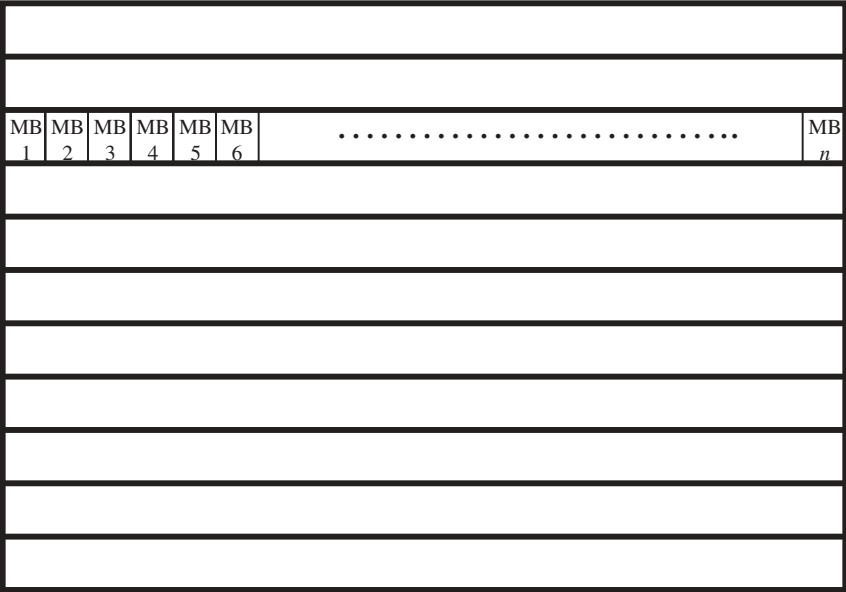


Figure 5.15 A row of macroblocks merges to form a slice.

decode the bit stream. This is usually caused by transmission errors affecting the bit stream received by the decoder. The digital bit stream is particularly vulnerable to transmission errors as a result of the large number of variable length Huffman code words used to reduce the amount of data that needs to be transmitted. This is illustrated in Figure 5.16 where a series of symbol is encoded using a simple Huffman code and then decoded after a single transmission error. The resulting decoded symbol stream is not the same as the transmitted symbol stream. Moreover, having some symbols decoded incorrectly, the total number of decoded symbols is also different.

When this occurs, the decoder often encounters some impossible situation such as attempting to decode more than 64 DCT coefficients in a block or trying to use a motion vector that points outside the reference picture. When this occurs, the decoder has no other alternative than to attempt to resynchronize to the incoming bit stream. This is achieved by looking for a resynchronization code word that can only occur in a nonerrored bit stream at the start of a slice (or at the start of a higher element in the syntax hierarchy as discussed a little later). Typically this is the 24-bit hexadecimal number 0x000001 and is called a start code. For this approach to work, care needs to be exercised in the design of the encoder syntax to ensure that the only time that a start code can occur is at the designated resynchronization points (i.e., 23 or more successive zero bits can never occur at other than the resynchronization point).

Because entire slices can be skipped in the same way as blocks and macroblocks, the slice header needs to contain information that allows the decoder to

Symbol	Huffman codeword
a	0
b	10
c	110
d	111

Encoded symbols	b	a	a	c	a	d
Transmitted Bit stream	10	0	0	110	0	111
Received Bit stream	10	<u>1</u>	0	110	0	111
Decoded symbols	b		b	c	a	d

Figure 5.16 The effect of a transmission error on Huffman decoding.

determine which slice in the current picture is about to be decoded. It would be possible to code this as an increment from the previous slice with a value of 1 indicating that the current slice is one on from the last slice. However, this would mean that when resynchronization was needed following a transmission error, the decoder could not be sure which slice was being decoded. For this reason (and noting that the number of slice headers is small), the actual slice number is incorporated into the slice header. Slice 1 is the first 16 lines of the picture, slice two is lines 17–32 and so on.

For resynchronization to be possible, it is essential that the decoder does not need to know any information that was transmitted before the current slice header (as this may have been lost). For this reason

- The current quantizer step size is included in the slice header.
- All elements that are usually coded with respect to a previous macroblock in the macroblock header are coded without prediction in the first macroblock after a slice header. This includes the macroblock address increment and the motion vectors.

A typical slice header is shown in Figure 5.17.

Start code 0000 0000 0000 0000 0000 0001	Slice number	Quantizer step size	First macroblock of slice
---	-----------------	------------------------	------------------------------

Figure 5.17 Typical slice header.

5.3.4. The Picture Layer

As we continue to move up the syntax hierarchy, the next level is the picture level. As indicated in Figure 5.15, the various slices merge to make up a complete picture, which is again preceded by a picture header. Resynchronization also needs to be available at the picture level, and so the picture header commences with a start code followed by a number that cannot be mistaken for a slice number (e.g., zero). This allows a picture header to be differentiated from a slice header by the decoder. As we shall see in the next chapter, the picture header can also be used to signal picture specific information. In order to minimize our syntax complexity, such information will be ignored here.

5.3.5. The Sequence Layer

The header at the sequence layer is used to signal information that applies to the entire video sequence that is to be decoded. Like the slice and picture layers, the sequence header begins with a start code followed by a unique number that differentiates it from slice and picture layer headers. Sequence specific information might include, inter alia, the following:

- The horizontal size of the pictures in the sequence (in pixels).
- The vertical size of the pictures in the sequence (in pixels).
- The picture rate (in pictures per second).
- The transmission bit rate (in bits per second).

5.4. BIT-STREAM SYNTAX

The task of specifying a bit-stream syntax is an important one because without a valid syntax, interoperability between encoders and decoders will be impossible to achieve. The syntax must be defined carefully—in the same way that a computer program needs to be specified carefully. This similarity has led to the use of a syntax description methodology that is quite similar to the C programming language. This section will provide an introduction to this method by using it to describe the simple video bit-stream syntax already developed in this chapter. These ideas will be extended in the next chapter when we consider the MPEG-2 video compression standard used for digital television broadcasting.

All video compression standards deal with the description of a decoder because it is mandated that all decoders that comply with the standard must be capable of decoding any compliant bit stream generated by an encoder. Although two different encoders presented with the same video material would most probably produce different encoded bit streams (both of which would comply with the standard), two different decoders presented with the same compliant bit stream should produce identical decoded outputs. The syntax is therefore written from the point of view of the decoding process.

5.4.1. Abbreviations

There are a number of abbreviations used to specify the type of data that is expected at the decoder at a particular point of the syntax. The most common of these are listed below.

bslbf	<i>bit string with the leftmost bit transmitted first</i>
uimsbf	<i>unsigned integer with most significant bit transmitted first</i>
vlcblf	<i>variable length code with leftmost bit transmitted first</i>

5.4.2. Start Codes

As indicated earlier, all start codes begin with the 24-bit hexadecimal code 0x000001. The syntax is written in such a way that this code will only occur at a start code in a nonerrored bit-stream. The next 8 bits (i.e, two hexadecimal numbers) specify the type of start code. As indicated earlier in this chapter, this is crucial information for the decoder particularly during resynchronization after a transmission error. The value corresponding to the various types of start codes described in this chapter are shown in Table 5.9. As we shall see later, the reserved start code types are used for other functions within the MPEG suite of standards. Slice start code field numbering is from 01 to AF hexadecimal (a total of 175 different values) with the number giving the vertical position of the slice within the picture. This means that there can be up to 2800 luminance lines per picture (175×16 rows per macroblock).

5.4.3. Describing the Bit-Stream Syntax

A number of constructs, similar to those of the C language, are used to describe the conditions under which the various syntactical units are present.

Table 5.9 Definition of start code types.

Type of start code	Start code type field (hexadecimal)
picture_start_code	00
slice_start_code	01 to AF
Reserved	B0 to B2
sequence_header_code	B3
Reserved	B4 to B6
sequence_end_code	B7
Reserved	B8
Reserved	B9 to FF

© This Table is based on AS/NZS 13818.2:2002. Permission to reprint has been granted by SAI Global Ltd. The standard can be purchased online at <http://www.sai-global.com>.

5.4.3.1. The While Construct

```
while (conditional_expression) {
    bitstream_element
    :
}
```

If the `conditional_expression` is true, the bit-stream elements occur next in the bit stream. The bit-stream elements are repeated until the expression is not true.

5.4.3.2. The Do-While Construct

```
do {
    bitstream_element
    :
} while (conditional_expression)
```

The `bitstream_elements` occur next in the bit stream and continue to be repeated until the `conditional_expression` is not true. Note that unlike the while construct, in the do-while construct the `bitstream_elements` must occur at least once.

5.4.3.3. The If-Else Construct

```
if (conditional_expression) {
    bitstream_element_a
    :
}
else {
    bitstream_element_b
    :
}
```

If the `conditional_expression` is true then the bit-stream elements beginning with `bitstream_element_a` occur next in the bit stream. If the `conditional_expression` is not true then the bit-stream elements beginning with `bitstream_element_b` occur next in the bit stream.

5.4.3.4. The For Construct

```
for(expression_1; conditional_expression; expression_2) {
    bitstream_element
    :
}
```

The for construct is simply a shorthand version of the while construct that is useful in many circumstances. The equivalent while construct would be

```

expression_1
while(conditional_expression) {
    bitstream_element
    :
    expression_2
}

```

In the for construct, `expression_1` is used to initialize the loop and is often the initial state of a counter. Before each iteration of the loop the `conditional_expression` is tested and the loop is terminated if the conditional expression is false. At the end of each iteration of the loop, `expression_2` is performed and is often used to increment a counter. A typical example of the use of the for construct is given below.

```

for (i=0; i<n; i++) {
    bitstream_element
    :
}

```

In this case, the bit-stream elements occur n times. The expression `i++` means that `i` is incremented by one.

5.4.4. Special Functions within the Syntax

A number of functions are used to define operations that need to be performed to correctly decode a bit stream.

5.4.4.1. The Bytealigned Function

The function `bytealigned()` returns the value 1 if the current position in the bit stream is on a byte (8-bit) boundary, that is, the next bit is the first bit of a byte. It is needed because start codes are typically bytealigned.

5.4.4.2. The Nextbits Function

The `nextbits()` function allows the next several bits in the bit stream to be compared to a particular bit string. This allows the syntax to move through the bit stream looking for a particular bit string. A typical example of this is when a start code is being sought.

5.4.4.3. The Next_Start_Code Function

The `next_start_code()` function shown in Table 5.10 uses both the `bytealigned()` and `nextbits()` functions described above to locate the next start code in the bit stream. Any zero bit stuffing (to reach the next byte boundary because start codes start on a byte boundary) and zero byte stuffing (which may have been added to increase the amount of transmitted data to meet rate control requirements) are ignored. This ensures that start codes can be preceded by any number of zero stuffing bits subject only to the start code being bytealigned.

Table 5.10 The next_start_code() function.

Syntax	Number of bits	Mnemonic
next_start_code() { while(!bytealigned()) { zero_bit } while(nextbits != '0000 0000 0000 0000 0000 0001') { zero_byte } }	1 8	"0" "0000 0000"

© This Table is based on AS/NZS 13818.2:2002. Permission to reprint has been granted by SAI Global Ltd. The standard can be purchased online at <http://www.sai-global.com>.

5.5. A SIMPLE BIT-STREAM SYNTAX

We will now define a simplified bit-stream syntax using the methods described in this chapter. This syntax greatly simplifies the understanding of the full MPEG-2 video syntax that is described in the next chapter. We will begin at the top layer (video sequence layer) and work our way down to the bottom layer (block layer).

5.5.1. The Video Sequence Layer

As explained earlier, this layer defines parameters that will apply to the entire video sequence. The syntax is given in Table 5.11.

Table 5.11 Video sequence syntax.

	Syntax	Bits	Mnemonic
1	video_sequence() {		
2	next_start_code()		
3	do {		
4	sequence_header()		
5	do {		
6	picture()		
7	}while (nextbits() == picture_start_code)		
8	}while (nextbits() == sequence_header_code)		
9	sequence_end_code	32	bslbf
10	}		

© This Table is based on AS/NZS 13818.2:2002. Permission to reprint has been granted by SAI Global Ltd. The standard can be purchased online at <http://www.sai-global.com>.

The function `next_start_code()` (line 2) is used to byte align the incoming bit stream. Next follows the `sequence_header()` (as defined below) followed immediately by a picture. Pictures then continue until `nextbits()` reveals that **picture_start_code** (line 7) is not present. The function `nextbits()` is then used to check if a new **sequence_header_code** is present (line 8). A new **sequence_header_code** followed by a new `sequence_header()` is possible before any picture of the sequence. This allows sequence parameters to be changed if necessary. If `nextbits()` reveals that there is neither a **picture_start_code** nor a **sequence_header_code**, then the sequence concludes with a **sequence_end_code**.

The function `sequence_header()` is defined in Table 5.12.

The sequence header() consists of

- (a) The 32-bit start code **sequence_header_code** (000001B3 hexadecimal).
- (b) The 12-bit horizontal size.
- (c) The 12-bit vertical size.
- (d) The 4-bit **aspect_ratio_information** defines the shape of each pixel in a picture. The horizontal and vertical size of the picture only define the aspect ratio of the picture if pixels can be assumed to be square. This is usually not the case. However, if we know the Display aspect ratio (DAR) and the horizontal and vertical size of the picture then the Sample aspect ratio (SAR) can be calculated according to the following equation.

$$\text{SAR} = \text{DAR} \times \frac{\text{horizontal_size}}{\text{vertical_size}}$$

The **aspect_ratio_information** defines either the SAR (for square pixels) or the DAR as shown in Table 5.13. The value 0000 is not used to avoid start code emulation.

Table 5.12 Sequence header syntax.

	Syntax	Bits	Mnemonic
1	<code>sequence_header() {</code>		
2	sequence_header_code	32	bslbf
3	horizontal_size	12	uimsbf
4	vertical_size	12	uimsbf
5	aspect_ratio_information	4	uimsbf
6	picture_rate	4	uimsbf
7	bit_rate	18	uimsbf
8	<code>next_start_code()</code>		
9	<code>}</code>		

© This Table is based on AS/NZS 13818.2:2002. Permission to reprint has been granted by SAI Global Ltd. The standard can be purchased online at <http://www.sai-global.com>.

Table 5.13 Definition of aspect_ratio_information.

aspect_ratio_information	Sample aspect ratio	Display aspect ratio
0000	Forbidden	Forbidden
0001	1.0 (Square)	—
0010	—	$3 \div 4$
0011	—	$9 \div 16$
0100	—	$1 \div 2.21$
0101 to 1111	—	Reserved

© This Table is based on AS/NZS 13818.2:2002. Permission to reprint has been granted by SAI Global Ltd. The standard can be purchased online at <http://www.sai-global.com>.

- (e) The 4-bit **picture_rate** defines the picture rate of the video sequence. Allowed values are given in Table 5.14. The value 0000 is not used to avoid start code emulation.
- (f) The 18 bit **bit_rate** field gives the bit rate of the channel in multiples of 400 bits/s and is assumed constant. Any bit rate that is not a multiple of 400 bits/s is rounded up to the next multiple of 400 bits/s. Thus the bit rate 999,800 bits/s would be rounded up to 1,000,000 bits/s and represented by a bit_rate value of 2500 ($1,000,000 \div 400$). The value 0x3FFFF is used to indicate variable bit-rate operation.

5.5.2. The Picture Layer

The picture layer provides information relating to the current picture only. An example of a simple syntax for this layer is given in Table 5.15.

Table 5.14 Allowable picture rates.

picture_rate	Pictures per second
0000	Forbidden
0001	$24,000 \div 1001 = 23.976$
0010	24
0011	25
0100	$30,000 \div 1001 = 29.97$
0101	30
0110	50
0111	$60,000 \div 1001 = 59.94$
1000	60
1001 to 1111	Reserved

© This Table is based on AS/NZS 13818.2:2002. Permission to reprint has been granted by SAI Global Ltd. The standard can be purchased online at <http://www.sai-global.com>.

Table 5.15 Picture syntax.

	Syntax	Bits	Mnemonic
1	picture() {		
2	picture_start_code	32	bslbf
3	temporal_reference	10	uimsbf
4	next_start_code()		
5	do {		
6	slice()		
7	}while (nextbits() == slice_start_code)		
8	}		

© This Table is based on AS/NZS 13818.2:2002. Permission to reprint has been granted by SAI Global Ltd. The standard can be purchased online at <http://www.sai-global.com>.

The header information at the picture layer consists of the following fields:

- (a) The 32-bit **picture_start_code** (00000100 hexadecimal).
- (b) A 10-bit **temporal_reference**, which is an unsigned integer that is incremented by 1 (modulo 1024) for each input picture.

The next_start_code() function is then called to look for a bytealigned start code that should be a slice header. Slices are then processed until the next start code is not a slice header.

5.5.3. The Slice Layer

The slice layer provides information about the current slice. This includes the quantizer step size to be used initially for the decoding of the slice. A simple slice syntax is given in Table 5.16.

Table 5.16 Slice syntax.

	Syntax	Bits	Mnemonic
1	slice() {		
2	slice_start_code	32	bslbf
3	quantizer_scale	5	uimsbf
4	do {		
5	macroblock()		
6	} while (nextbits() != 000 0000 0000 0000 0000 0000)		
7	}		

© This Table is based on AS/NZS 13818.2:2002. Permission to reprint has been granted by SAI Global Ltd. The standard can be purchased online at <http://www.sai-global.com>.

The slice begins with a **slice_start_code**. The first 24-bits are the hexadecimal code 000001 whereas the last 8 bits give the vertical position of the slice and take values in the range 01 to AF hexadecimal.

This is followed by the **quantizer_scale** that defines the quantizer step size to be used for the quantization of DCT coefficients. This is a 5-bit number in the range 1–31 that corresponds to quantizer step sizes in the range 2–62, respectively. This quantizer value is used for the entire slice unless changed in a subsequent macroblock header.

Macroblocks are then processed (lines 4–6) until a start code (23 successive zeros) is encountered indicating a new slice, picture or sequence level start code.

5.5.4. The Macroblock Layer

The macroblock layer provides detailed information about each 16 × 16 block of luminance (and associated chrominance) pixels. This includes the way that the macroblock is to be coded (intra, inter, or motion compensated), a new quantizer step size (if needed) and motion vector (if required). A simplified syntax for the macroblock layer is shown in Table 5.17.

The first piece of information in the macroblock (lines 2–4) is the **macroblock_address_increment**. This indicates the difference between the current macroblock address and the address of the previously coded macroblock. The code word **macroblock_address_increment** is limited to the range 1–33. Because the

Table 5.17 Macroblock syntax.

	Syntax	Bits	Mnemonic
1	macroblock() {		
2	while(nextbits() == '0000 0001 000') {		
3	macroblock_escape	11	vlclbf
4	macroblock_address_increment	1–11	vlclbf
5	macroblock_type	1–6	vlclbf
6	if(macroblock_quant) {		
7	quantizer_scale }	5	uimsbf
8	if(macroblock_motion) {		
9	motion_horizontal_code	1–11	vlclbf
10	motion_vertical_code	1–11	vlclbf
11	}		
12	if(macroblock_pattern) {		
13	coded_block_pattern	3–9	vlclbf
14	for(i=0; i<6; i++) {		
15	block(i) }		
16	}		

© This Table is based on AS/NZS 13818.2:2002. Permission to reprint has been granted by SAI Global Ltd. The standard can be purchased online at <http://www.sai-global.com>.

Table 5.18 Definition of Internal Syntax Variable Based on **macroblock_type**.

macroblock_type	1	01	001	0001 1	0001 0	0000 1	0000 01
Macroblock type	MC	Inter	MC	Intra	MC	Inter	Intra
Coded	Yes	Yes	No	Yes	Yes	Yes	Yes
New quantizer	No	No	No	No	Yes	Yes	Yes
macroblock_quant	0	0	0	0	1	1	1
macroblock_motion	1	0	1	0	1	0	0
macroblock_pattern	1	1	0	0	1	1	0
macroblock_intra	0	0	0	1	0	0	1

© This Table is based on AS/NZS 13818.2:2002. Permission to reprint has been granted by SAI Global Ltd. The standard can be purchased online at <http://www.sai-global.com>.

previously coded macroblock may be more than 33 macroblocks back, the **macroblock_escape** code word can be used to allow for larger increments. Each time the **macroblock_escape** code word is included 33 is added to the total increment. Thus an increment of 70 macroblocks would comprise the **macroblock_escape_code word** twice followed by a **macroblock_address_increment** of four macroblocks.

Next (line 5) comes the **macroblock_type** code word. As discussed earlier in this chapter, this defines which other elements are contained in the macroblock. Based on the value of the **macroblock_type** code word, various internal syntax variables are set as shown in Table 5.18.

The meaning of these internal variables is listed below.

- **macroblock_quant** when set indicates that the macroblock contains a new value for **quantizer_scale**.
- **macroblock_motion** when set indicates that the macroblock contains a motion vector.
- **macroblock_pattern** when set indicates that the macroblock contains a coded block pattern.
- **macroblock_intra** when set indicates that the macroblock is coded in intramode.

It is now a simple task to interpret the remainder of the macroblock syntax. If **macroblock_quant** is set (line 6) then a new 5 bit fixed length value of **quantizer_scale** appears next in the bit stream (line 7).

If **macroblock_motion** is set (line 8) then the two components of the motion vector appear next in the bit stream (lines 9–10). Note that the motion vector difference between the motion vector in the current macroblock and the motion vector in the previous macroblock is transmitted. The prediction motion vector is reset to zero if:

- the macroblock immediately follows a slice header.
- the previous macroblock was coded in intermode because a zero motion vector is assumed in this case. This applies also if the previous macroblock was a skipped macroblock.

- the previous macroblock was coded in intramode because no motion vector information is available from an intracoded macroblock.

The motion vector differences are variable length coded as described earlier in this chapter and are of length 1–11 bits each.

If **macroblock_pattern** is set (line 13) then the coded block pattern appears next in the bit stream (line 14).

We now have all the information needed to decode the blocks of DCT coefficients associated with the macroblock, and this is now performed (lines 14–15) for each of the six possible blocks in a macroblock.

5.5.5. The Block Layer

A simple syntax for the block layer is given in Table 5.19.

The coded block pattern read at the macroblock level produces a 6-bit pattern code indicating which of the blocks 0 (Y_0) to 5 (V) are to be coded with a value of 1, indicating that the block is to be coded (see Table 5.8). If the block is not coded, as indicated by **pattern[i]** (line 2) then the block syntax is skipped as required for a skipped block. Next a check is made to see if the block is coded in intramode

Table 5.19 Block syntax.

	Syntax	Bits	Mnemonic
1	block(i) {		
2	if(pattern_code[i]) {		
3	if (macroblock_intra) {		
4	if (i < 4) {		
5	dct_dc_size_luminance	2–7	vlclbf
6	if (dc_size_luminance !=0) {		
7	dct_dc_differential}	1–8	uimsbf
8	}		
9	else {		
10	dct_dc_size_chrominance	2–8	vlclbf
11	if (dc_size_chrominance != 0) {		
12	dct_dc_differential }	1–8	uimsbf
13	}		
14	}		
15	while (nextbits() != ‘10’) {		
16	dct_coeff_next }	3–28	vlclbf
17	end_of_block	2	vlclbf
18	}		
19	}		

© This Table is based on AS/NZS 13818.2:2002. Permission to reprint has been granted by SAI Global Ltd. The standard can be purchased online at <http://www.sai-global.com>.

(line 3). If this is the case then the DC DCT coefficient is treated separately from the other DCT coefficients. As described earlier, each DC coefficient is predicted from an earlier DC coefficient. For the first four blocks where the block count, i , is in the range of 0–3 (line 4), the variable length codes for **dct_dc_size_luminance** and **dct_dc_differential** appear next in the bit stream (lines 5–7) whereas for a block count in the range 4–5 the **dct_dc_size_chrominance** and **dct_dc_differential** appear next in the bit stream. Then follows the remaining (run,level) pairs of DCT coefficients (in the case of intra macroblocks) or all DCT coefficients represented as (run,level) pairs (in the case of motion-compensated or inter macroblocks) until the end of block code word (“10”) is encountered.

5.6. CONCLUSION

This chapter has provided an introduction to the syntax used by video encoders to transmit compressed video information to the decoder in a form that it can understand. We have also introduced a number of tools for the efficient representation of this information.

In the next chapter, detailed consideration will be given to the MPEG-2 video compression standard that forms the basis of all current digital television broadcasting systems worldwide.

PROBLEMS

- 5.1 Briefly explain the reasons that a bit-stream syntax is required for video encoders. What are the major advantages?
- 5.2 Most of the information contained in a video bit stream is encoded using variable word length codes. Why is this the case? What disadvantages does the use of these variable word length codes introduce? How does the use of a video encoder syntax allow these problems to be minimized?
- 5.3 A camera produces signals representing the red, green, and blue components of the scene that it is imaging. This information is invariably transformed into *YUV* form. Explain briefly why this is the case.
- 5.4 Table 5.1 and Figure 5.6 show that the (run,level) pairs representing quantized DCT coefficients are sometimes represented by variable length code words and sometimes by fixed length code words. Given that even in the (run,level) pairs represented by the fixed length code words some will be more common than others, it would seem desirable to use variable length codes here as well. Explain briefly why this is not done.
- 5.5 Why is it that the macroblock header for an intracoded macroblock does not require a field for the coded block pattern?
- 5.6 Explain briefly why motion vectors are coded differentially. What are the advantages and disadvantages of doing this?
- 5.7 A quantized block of DCT coefficients from a macroblock coded in intramode is given in Figure 5.18. Calculate the code words that would need to be transmitted to represent this block of data. Assume that this is the first block in the macroblock and that the macroblock immediately follows a slice header.

+ 20	0	0	0	0	0	0	0
0	0	- 1	0	+ 1	0	- 1	0
0	0	0	0	0	0	0	0
0	0	+ 1	0	0	- 1	0	0
0	0	0	- 1	0	0	0	0
0	0	0	- 1	- 1	0	+ 1	0
0	0	0	+ 1	0	0	- 1	0
0	0	0	0	0	0	0	0

Figure 5.18 Quantized DCT coefficients for Problem 5.7.

5.8 A quantized block of DCT coefficients from a macroblock coded using motion compensated prediction is given in Figure 5.19. Calculate the code words that would need to be transmitted to represent this block of data.

- 2	0	0	0	0	0	0	0
0	- 5	0	- 1	0	0	0	0
+ 13	- 7	+ 2	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	- 1	0	0	0	0	0
0	0	0	0	0	0	0	0
- 1	+ 2	0	0	0	0	+ 1	0
- 3	0	0	0	0	0	0	0

Figure 5.19 Quantized DCT coefficients for Problem 5.8.

5.9 The seventh macroblock of a particular slice is to be coded. The macroblock is coded using motion compensated prediction and the motion vector employed is $(+2, -1)$. After motion-compensated prediction, application of the DCT, and quantization with the same quantizer as used in the previous macroblock, all blocks in the macroblock contain all zero coefficients with the exception of block Y_2 that is shown in Figure 5.20.

+ 2	0	0	0	0	0	0	0
0	- 1	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

Figure 5.20 Quantized DCT data for Problem 5.9.

The previous macroblock coded was the fourth macroblock in the slice and was also coded with motion-compensated prediction using a motion vector of (+1, 0). Using the syntax described in this chapter, determine the bit stream required to represent this macroblock.

5.10 A macroblock is coded in intramode. The first quantized block (Y_1) in the macroblock contains only a quantized DC DCT coefficient with value 120. The second block (Y_2) is shown in Figure 5.21.

+ 95	+ 1	+ 1	0	0	0	0	0
- 1	- 1	- 1	0	0	0	0	0
+ 1	+ 1	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

Figure 5.21 Quantized DCT data for Problem 5.10.

Using the syntax described in this chapter, determine the bit stream required to represent this block.

5.11 The two-dimensional DCT of a luminance block in an intracoded macroblock produces the data shown in Figure 5.22.

+ 755.6	+ 141.3	+ 78.6	+ 29.1	+ 2.1	- 7.6	- 2.7	0.9
- 119.2	- 177.1	- 107.2	+ 39.6	+ 21.2	- 18.0	- 8.1	0.0
+ 107.5	+ 136.7	+ 54.9	- 42.1	- 32.0	+ 6.4	- 1.5	0.0
- 45.0	- 41.8	+ 30.8	+ 67.6	+ 21.3	+ 2.6	+ 5.0	+ 3.7
- 1.1	- 13.7	- 32.9	- 20.5	+ 4.4	+ 7.6	+ 3.2	+ 0.6
+ 15.1	+ 15.4	+ 8.8	- 15.9	- 23.0	- 8.4	- 5.0	0.0
- 13.8	- 22.0	+ 1.3	+ 13.7	- 5.8	- 11.7	- 8.9	+ 0.9
+ 14.2	+ 14.5	+ 8.2	+ 1.2	- 10.4	0.0	+ 6.5	+ 4.0

Figure 5.22 DCT data for Problem 5.11.

- (a) If the **quantizer_scale** value is 10 and the DC DCT coefficient of the previous luminance block was 480, calculate the values of the quantized DCT coefficients.
- (b) Determine the bit stream required to represent these quantized DCT coefficients using the syntax developed in this chapter.

5.12 The bit stream shown in Figure 5.23 represents the first block of an intracoded macroblock which immediately follows a slice header.

1101010000000000011111101000000101110

Figure 5.23 Bit stream values for Problem 5.12.

- (a) Determine the decoded quantized DCT values.
 - (b) If the quantizer step size used was 8, determine the DCT values after inverse quantization.
 - (c) Determine the decoded pixel values for this block.
- 5.13 The first five motion vectors in a slice are shown in Figure 5.24.

(+4,-7) (+5,-9) (+2,-1) (+2,+2) (+3,-1)

Figure 5.24 Motion vectors for Problem 5.13.

- (a) Calculate the differential motion vectors that need to be transmitted to the receiver.
 - (b) Calculate the code words used to represent these motion vectors.
- 5.14 Repeat Problem 5.13 for the case where the motion vectors given in Figure 5.24 apply to macroblocks 1, 2, 4, 5, and 6, respectively. Macroblock 3 is a skipped macroblock.
- 5.15 The DC DCT coefficients for two successive macroblocks coded in intramode are shown in Figure 5.25. Determine the variable length code words that would be transmitted to represent the DC coefficients in the second macroblock.

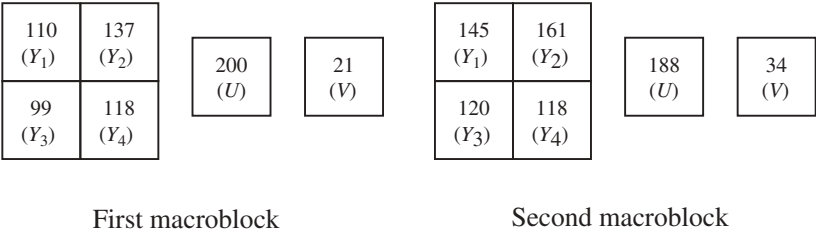


Figure 5.25 DC DCT coefficient values for Problem 5.15.

- 5.16 Repeat Problem 5.15, but this time assume that the first macroblock in Figure 5.25 is preceded by a slice header. Calculate the variable wordlength code words that would be used to represent each of the DC coefficients in both macroblocks.
- 5.17 Explain the meaning of the following piece of syntax shown in Table 5.20. What functionality does this syntax allow?

Table 5.20 Syntax for Problem 5.17.

	Syntax	Bits	Mnemonic
1	while (nextbits() == '1'){		
2	extra_bit_picture /* with value 1 */	1	uimsbf
3	extra_information_picture	8	uimsbf
4	}		
	extra_bit_picture /* with value 0 */	1	uimsbf

5.18 Suppose that it was decided to always transmit the DC DCT coefficients in all blocks (as opposed to just intrablocks).

- (a) Rewrite the block syntax shown in Table 5.19 to reflect this change.
- (b) Would this change have any impact on any of the higher level syntax elements?

5.19 If we are to be able to decode a bit stream stored in a file using MATLAB, we need to be able to read bits from the file. The MATLAB command given below reads one byte from the file defined by fid.

```
fread(fid,1,'uchar');
```

Develop a MATLAB function bit_in(length) that will read a code word of “length” bits from the file. For example, suppose that the file contained the first four bytes shown in Figure 5.26.

00011100 10010100 01110011 10010010

Figure 5.26 Data for Problem 5.19.

The succession of function calls listed below would then return the binary code words shown in Figure 5.27.

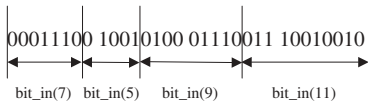


Figure 5.27 Reading variable length code words from a file.

bit_in(7) would return 0001110

bin_in(5) would return 01001

bit_in(9) would return 010001110

bit_in(11) would return 01110010010

This is illustrated in Figure 5.27.

5.20 An important part of the video syntax is the nextbits() function that allows the next several bits of the bit stream to be checked for a particular value. Write and test a MATLAB function that will carry out this function. Note that your routine needs to be able to check the next several bits in the bit stream while still leaving the bits in the bit stream to be subsequently read by another part of the syntax if required. Line 15 of the block syntax shows an example of where this might be necessary. Note that the MATLAB functions fseek and ftell might prove useful in carrying out this task.

MATLAB EXERCISE 5.1: EFFICIENT CODING OF MOTION VECTOR INFORMATION

In this chapter we saw an example of how the coding of motion vectors using horizontal prediction. In this exercise, you will verify that these results are correct and study the efficiency of this approach for other video sequences.

Section 1 Coding of motion vectors for a video sequence

- (a) Load the first and fifth picture from a video sequence. Form the motion-compensated prediction of the fifth picture using the first picture as the reference picture using a search range of ± 8 both horizontally and vertically. Save the motion vectors generated.
- (b) Display the histogram of each component of the motion vector information.
- (c) Predict each component of each motion vector by the same component of the motion vector immediately to its left. The first motion vector of each row of macroblocks should be predicted using a zero motion vector. Draw the histogram of the prediction differences for each motion vector component.
- (d) Compare the entropy of the original motion vector information and the entropy of the motion vector prediction differences. Comment on the result.

Section 2

Repeat Section 1 for pairs of pictures taken from other video sequences. Comment on the results achieved. What types of sequences achieve the biggest saving when motion-compensated prediction is employed?

Section 3

Extend the motion-compensated prediction process developed in Section 1 to include motion-compensated prediction to half pixel accuracy. Use your improved motion-compensated prediction scheme for various pairs of pictures from the sequences provided. Comment on the performance compared to motion-compensated prediction to single pixel accuracy.

MATLAB EXERCISE 5.2: A SIMPLE VIDEO ENCODER

In this exercise you are required to write an encoder that can be used to code pictures in a video sequence. For simplicity, we will code every macroblock in every picture in intramode. Although not terribly effective from a coding efficiency point of view, this exercise will give you experience in working with a reasonably detailed syntax. In running the encoder, you can assume the following.

- (a) The shape of each pixel in the picture is square (because this is almost certainly the case for your computer monitor).
- (b) For constant bit-rate operation, it is necessary to include rate control into the encoder. This is too complex in this case so constant quantizer, variable bit-rate operation should be employed.

- (c) The quantizer step size is to remain the same for each picture in the sequence.
- (d) If only luminance information is available, the chrominance blocks should be assumed to be always zero.

Use your encoder to encode a variety of short video sequences. Comment on the quality and bit rate requirement for each sequence over a range of quantizer step sizes.

MATLAB EXERCISE 5.3: A SIMPLE VIDEO DECODER

In this exercise you will decode a video sequence that has been encoded using the syntax described in this chapter. Given that the full syntax presented is quite complex, we will simplify the task by assuming that all macroblocks in all pictures are coded in intramode. This means that there will be only one set of variable length code words in the bit stream for the (run,level) pairs of quantized DCT coefficients. The variable word length codes for motion vector differences, macroblock increment, and coded block pattern are not needed in intra macroblocks. The bit stream to be decoded is one of the bitstreams that you generated in MATLAB Exercise 5.2.

Display the decoded sequence on your computer monitor using the MATLAB movie function.

MATLAB EXERCISE 5.4: A VIDEO ENCODER

In this exercise you are required to write an encoder that can be used to code pictures in a video sequence. In running the encoder, you can assume the following.

- (a) The first picture in the sequence is coded in intramode with all the remaining pictures coded using motion-compensated prediction from the previous picture.
- (b) All macroblocks in a picture that uses motion-compensated prediction are coded using motion-compensated prediction (i.e., intramode and intermode are not used for these pictures).
- (c) The shape of each pixel in the picture is square (because this is almost certainly the case for your computer monitor).
- (d) For constant bit-rate operation, it is necessary to include rate control into the encoder. This is too complex in this case so constant quantizer, variable bit-rate operation should be employed.
- (e) The quantizer step size is to remain the same for each picture in the sequence.
- (f) The motion vector search range should be ± 8 pixels both horizontally and vertically.
- (g) If only luminance information is available, the chrominance blocks should be assumed to be always zero.

Use your encoder to encode a variety of short video sequences. Comment on the quality and bit rate requirement for each sequence over a range of quantizer step sizes.

This exercise is very challenging and requires significant effort.

MATLAB EXERCISE 5.5: A VIDEO DECODER

In this exercise you will decode a video sequence that has been encoded using the syntax described in this chapter. The bit stream to be decoded is one of the bit streams generated by your encoder in MATLAB Exercise 5.4. As might be expected, all macroblocks in the first picture of the sequence are encoded in intramode. Display the decoded sequence on your computer monitor using the MATLAB movie function.

Note that this exercise is quite challenging and requires significant effort.

MATLAB EXERCISE 5.6: INTRA/INTER/MOTION-COMPENSATED CODING OF MACROBLOCKS

In this chapter we saw that macroblocks can be coded in either intramode, intermode, or motion-compensated mode. It is, of course, necessary for the encoder to determine the appropriate manner of encoding for each macroblock. Like many encoder issues, this is not the subject of standardization because decoders only need to know the result of the choice and this is specified in the **macroblock_type** field. This gives encoder manufacturers the ability to differentiate their products in the marketplace because the more efficiently this choice is made, the better will be the performance of the encoder. In this exercise you will look at the success of a simple technique for deciding which of the three modes should be chosen. For this exercise, you will need first to have completed MATLAB Exercise 4.3.

The choice is made in two steps. First a decision is made whether to code in intermode or using motion compensation. Following this, a decision is made as to whether to use the inter/MC decision made in the first step or to use intracoding. The approach described was used in the development of the MPEG video standard.

Stage 1 Inter – motion-compensated decision

Calculate the average absolute difference for the luminance macroblock using the chosen motion vector (M) and using the zero motion vector (Z). The decision as to whether to use intermode or motion compensated mode is made according to Figure 5.28. Points on the line dividing the two regions are defined as being inter.

Stage 2 Intra – nonintra decision

The mode chosen in stage 1 (inter or MC) needs to be compared with intracoding. This is done by calculating the variance of the luminance information in the macroblock to be encoded (**var_current**) and the variance of the luminance difference macroblock produced after the appropriate prediction macroblock is subtracted (**var_difference**). Note that the macroblock mean luminance value needs to be

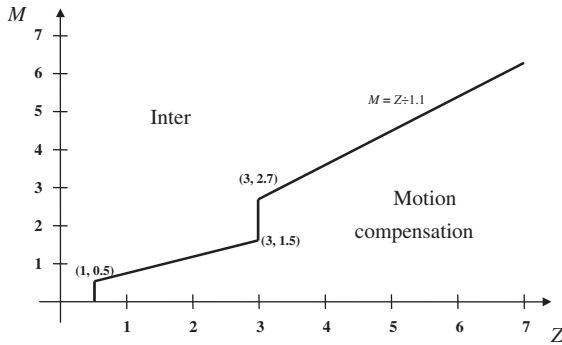


Figure 5.28 Inter motion-compensated decision.

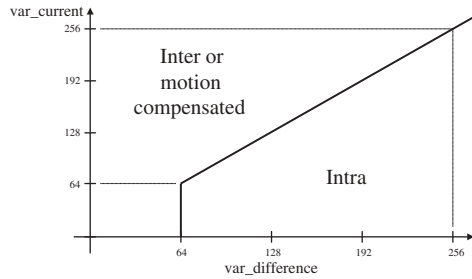


Figure 5.29 Intra – Inter/MC Decision.

subtracted from the current macroblock before var_current can be calculated. The difference macroblock is always assumed to have a zero mean for the calculation of var_difference . The decision between intra or inter/MC coding is then made according to Figure 5.29.

Modify the encoder developed in MATLAB Exercise 5.4 to incorporate this method of decision between inter – motion compensated – intra macroblocks. Apply the encoder to a variety of short video sequences. Comment on the change in performance compared to the encoder of MATLAB Exercise 5.4.